

Sistema de Recomendacion de Centros de Salud Preventiva Basado en Capacidad Operativa e Indicadores Climaticos para la Region Puno

Informe tecnico de implementacion funcional

Unidad academica: Ingenieria de Sistemas

Nivel: X Semestre

Dominio: Salud publica regional

Region: Puno, Peru

Anio: 2025

Resumen ejecutivo

El proyecto desarrollo un sistema funcional de recomendacion para orientar consultas preventivas hacia establecimientos de salud de la region Puno. La solucion integro una API REST en FastAPI, persistencia relacional, autenticacion JWT, un motor de filtrado basado en contenido, una interfaz React con mapa interactivo y un notebook reproducible para analisis exploratorio y evaluacion. La decision de recomendacion combino coincidencia de servicio, categoria IPRESS, distancia geografica, capacidad operativa y penalidad por alerta climatica. El sistema evita datos clinicos individuales y se diseno bajo principios de minimizacion, finalidad y trazabilidad.

Palabras clave: sistemas de recomendacion, salud preventiva, RENIPRESS, SENAMHI, FastAPI, filtrado basado en contenido, Puno.

Índice

1.- Título del proyecto	1
2.- Planteamiento del problema	1
Descripcion del problema	1
Contexto	2
Necesidad del sistema de recomendacion	2
3.- Objetivos	3
Objetivo general	3
Objetivos especificos	3
4.- Justificacion	4
5.- Fases del prototipo	4
5.1.- Fase 1: Inicial	4
Definicion del dominio	4
Identificacion de usuarios, items e interacciones	4
5.2.- Fase 2: Analisis exploratorio de datos (EDA)	5
Fuente de datos y descripcion del dataset	5
Limpieza de datos	6
Transformaciones realizadas	6
Analisis exploratorio	6
5.3.- Fase 3: Implementacion del modelo de recomendacion	7
Tecnica utilizada	7
Algoritmos implementados	7
Generacion de Top-N recomendaciones	8
5.4.- Fase 4: Evaluacion del modelo	8
Herramientas	9
6.- Desarrollo del sistema	9
Descripcion tecnica del sistema	9
Arquitectura	10
Componentes clave	10
Flujo de funcionamiento	11
Contrato de API	13
Implementacion en notebook o script	13
Criterios de calidad	14

7.- Resultados	14
Estadísticas del dataset	14
EDA	14
Ejemplos de recomendacion	14
Metricas	14
Metricas de rendimiento y calidad operacional	15
Sistema funcionando	16
Despliegue y reproducibilidad	16
8.- Conclusiones	16
9.- Recomendaciones	17
Sustento tecnico de las recomendaciones	17
Etica, privacidad y gobernanza	18
Análisis de vulnerabilidades y riesgos	18
Plan de endurecimiento	20
10.- Referencias	21
11.- Anexos	23
Anexo A: Código fuente	23
Anexo B: Dataset	23
Anexo C: Evidencias adicionales	23

1.- Titulo del proyecto

Sistema de Recomendacion de Centros de Salud Preventiva Basado en Capacidad Operativa e Indicadores Climaticos para la Region Puno.

El proyecto corresponde al desarrollo de un prototipo funcional de sistema de recomendacion aplicado al dominio de salud publica regional. La propuesta integra datos de establecimientos de salud, atributos operativos, servicios preventivos, ubicacion geografica y alertas climaticas para generar recomendaciones Top-N de centros de atencion preventiva en la region Puno. El sistema fue disenado como una aplicacion web completa, compuesta por backend REST, base de datos relacional, motor de recomendacion, frontend interactivo, notebook analitico e informe tecnico.

El titulo refleja tres decisiones de disenio. Primero, el sistema recomienda *centros de salud preventiva*, por lo que no diagnostica enfermedades ni reemplaza la decision clinica. Segundo, el ranking se basa en *capacidad operativa*, entendida como categoria IPRESS, disponibilidad de servicios, estado operativo y capacidad aproximada. Tercero, incorpora *indicadores climaticos*, debido a que las heladas, precipitaciones intensas y granizadas pueden afectar el traslado de los ciudadanos en zonas altoandinas. En consecuencia, la recomendacion no solo busca similitud entre necesidad e item, sino tambien accesibilidad realista.

2.- Planteamiento del problema

Descripcion del problema

La region Puno presenta dispersion territorial, zonas rurales de dificil acceso, variacion altitudinal elevada y exposicion frecuente a eventos climaticos adversos. Estas condiciones afectan el acceso oportuno a servicios preventivos como vacunacion, control prenatal, control de crecimiento y desarrollo, tamizaje, nutricion, odontologia y planificacion familiar. Aunque existen establecimientos registrados en RENIPRESS/SUSALUD, la informacion disponible para el ciudadano suele estar distribuida, no siempre se consulta de forma integrada y no incorpora de manera directa el riesgo climatico del momento.

El problema central es la falta de un mecanismo digital que oriente al ciudadano hacia establecimientos preventivos adecuados considerando, al mismo tiempo, oferta de servicios, distancia, categoria del establecimiento, estado operativo y alerta climatica. Sin esa integracion, la eleccion del establecimiento depende de conocimiento local, recomendaciones informales o cercania aparente. Esta situacion puede producir desplazamientos innecesarios, saturacion diferencial de algunos establecimientos, perdida de oportunidad preventiva y mayor exposicion a condiciones climaticas peligrosas.

Desde la perspectiva de sistemas de recomendacion, el desafio consiste en ordenar items disponibles (establecimientos) para un usuario que expresa una necesidad concreta (tipo de

atencion preventiva) bajo restricciones geograficas y ambientales. A diferencia de dominios comerciales o de entretenimiento, el error de recomendacion puede afectar tiempo, costo de traslado y seguridad. Por ello, el algoritmo debe ser explicable, auditable y prudente.

Contexto

El contexto del prototipo es la salud publica regional. El sistema esta orientado a ciudadanos de Puno que requieren informacion para acceder a atencion preventiva. El sector salud peruano ha documentado brechas de acceso y continuidad de servicios, especialmente en regiones con barreras geograficas ([Ministerio de Salud del Perú, 2023](#)). La transformacion digital en salud se considera una via para reducir desigualdades si se implementa con criterios de inclusion, interoperabilidad y gobernanza ([Organización Panamericana de la Salud, 2023](#)).

Las fuentes de referencia son RENIPRESS/SUSALUD para establecimientos de salud y SENAMHI para informacion meteorologica regional. RENIPRESS permite identificar IPRESS autorizadas y atributos basicos del establecimiento ([Superintendencia Nacional de Salud, 2026](#)); SENAMHI publica informacion meteorologica util para valorar condiciones de traslado ([Servicio Nacional de Meteorología e Hidrología del Perú, 2026](#)). El prototipo usa estas fuentes como base conceptual y operativa, complementadas con datos reproducibles de ejemplo para ejecucion local.

Necesidad del sistema de recomendacion

La necesidad del sistema se justifica porque el ciudadano no solo requiere una lista de establecimientos, sino un ranking contextualizado. Un listado simple no diferencia adecuadamente entre establecimientos cercanos pero sin el servicio solicitado, establecimientos con buena capacidad pero lejos del usuario, o establecimientos en zonas con alerta climatica activa. Un recomendador permite combinar varios criterios en una unica salida ordenada y explicable.

La tecnica seleccionada fue filtrado basado en contenido porque el proyecto no dispone inicialmente de historiales reales de interaccion. Segun [Ricci et al. \(2022\)](#), este enfoque es apropiado cuando se cuenta con atributos descriptivos de los items y se requiere recomendar sin depender de calificaciones previas. En salud publica, esta decision tambien reduce riesgos de privacidad, ya que no exige historiales clinicos individuales.

3.- Objetivos

Objetivo general

Desarrollar un sistema de recomendacion funcional que permita orientar a ciudadanos de la region Puno hacia establecimientos de salud preventiva, utilizando filtrado basado en contenido, capacidad operativa, distancia geografica e indicadores climaticos.

Objetivos especificos

- Implementar una tecnica de recomendacion basada en contenido para generar Top-N establecimientos preventivos.
- Analizar y transformar datos de establecimientos, servicios, coordenadas y alertas climaticas mediante un notebook reproducible.
- Desarrollar una API REST segura con autenticacion JWT, persistencia relacional y endpoints de consulta, historial y administracion.
- Construir una interfaz web funcional que consuma la API real, permita autenticacion, registre consultas y presente resultados en tarjetas, graficos y mapa interactivo.
- Evaluar el sistema mediante metricas de recomendacion, pruebas automatizadas y analisis de riesgos tecnicos.

Cuadro 1: Objetivos especificos y evidencia de implementacion.

Objetivo especifico	Evidencia tecnica
Integrar datos abiertos de establecimientos y clima	Servicios <code>renipress_service.py</code> y <code>clima_service.py</code> ; <code>seed</code> reproducible.
Implementar recomendacion explicable	Motor <code>recommender.py</code> con similitud del coseno, distancia y penalidad.
Exponer API segura	FastAPI, JWT, hashing bcrypt y rutas protegidas.
Presentar resultados al ciudadano	React, Leaflet, tarjetas de score, grafico y banner climatico.
Documentar y evaluar el modelo	Notebook con EDA, metricas y serializacion.

4.- Justificacion

El sistema es importante porque integra informacion publica dispersa en una herramienta operativa de apoyo a la decision ciudadana. En un entorno altoandino, la distancia y el clima no son variables secundarias: pueden definir si una persona puede trasladarse con seguridad o si conviene priorizar un establecimiento alternativo. Por esa razon, el prototipo aporta valor al combinar datos sanitarios y meteorologicos en un unico ranking.

Los beneficios esperados se agrupan en tres niveles. Para el ciudadano, el sistema reduce incertidumbre al mostrar recomendaciones ordenadas, distancia aproximada, servicios disponibles, horario, score y alerta climatica. Para la gestion publica, el historial de consultas puede servir como insumo agregado para detectar demanda preventiva por territorio, siempre bajo criterios de anonimato y minimizacion. Para el ambito academico, el proyecto demuestra una arquitectura completa de recomendacion desplegable, no limitada a una visualizacion estatica.

La aplicabilidad en el mundo real es alta porque el diseno utiliza tecnologias abiertas y ampliamente usadas: FastAPI, SQLAlchemy, PostgreSQL, React, Leaflet, Pandas y Scikit-learn. Ademas, evita depender de datos clinicos individuales. Esta decision se alinea con la Ley N. 29733 y su reglamento ([Congreso de la República del Perú, 2011](#); [Ministerio de Justicia y Derechos Humanos, 2013](#)), al reducir la exposicion de datos sensibles y trabajar con datos operativos de establecimientos. La literatura reciente senala que los recomendadores en salud deben priorizar seguridad, explicabilidad y evaluacion contextual, no solo exactitud matematica ([Sun et al., 2023](#); [Akbari and Mahdipour, 2023](#)).

5.- Fases del prototipo

5.1.- Fase 1: Inicial

Definicion del dominio

El dominio seleccionado fue salud publica preventiva en la region Puno. Los servicios considerados incluyen vacunacion, control prenatal, control de crecimiento y desarrollo, tamizaje, odontologia, psicologia, nutricion y planificacion familiar. La eleccion del dominio responde a una necesidad publica concreta: orientar consultas preventivas antes de que el ciudadano realice desplazamientos costosos o riesgosos.

Identificacion de usuarios, items e interacciones

Los usuarios del sistema son ciudadanos que requieren atencion preventiva. No se modelan como pacientes clinicos ni se registran diagnosticos. Los items son establecimientos de salud de categorias I-1 a II-2, con atributos como categoria, distrito, provincia, coordenadas,

horario, servicios disponibles, estado operativo y capacidad aproximada. El tipo de interaccion principal es una consulta preventiva: el usuario indica ubicacion, edad, tipo de atencion, distancia maxima y numero de recomendaciones.

Cuadro 2: Elementos del dominio del prototipo.

Elemento	Definicion en el sistema
Usuarios	Ciudadanos de Puno que buscan orientacion para atencion preventiva.
Items	Establecimientos de salud registrados o compatibles con RENIPRESS/SUSALUD.
Interaccion	Consulta autenticada que genera recomendaciones y queda persistida.
Contexto	Ubicacion del usuario, distancia maxima, tipo de atencion y alerta climatica.
Salida	Ranking Top-N con score, distancia, servicios, categoria y nivel de alerta.

5.2.- Fase 2: Analisis exploratorio de datos (EDA)

Fuente de datos y descripcion del dataset

El prototipo utilizo datos compatibles con RENIPRESS/SUSALUD y SENAMHI. Para ejecucion reproducible se incluyeron archivos semilla con establecimientos de Puno, servicios preventivos y alertas climaticas. El dataset contiene variables de identificacion, ubicacion, categoria, servicios, capacidad, horario, estado operativo y nivel de alerta. Esta decision permite ejecutar el proyecto sin depender de disponibilidad externa inmediata, manteniendo una estructura equivalente a las fuentes oficiales.

Cuadro 3: Diccionario resumido de datos principales.

Entidad	Campo	Uso en el sistema
Establecimiento	codigo_renipress	Identificador unico para trazabilidad.
Establecimiento	categoria	Codificacion one-hot del perfil de capacidad.
Establecimiento	latitud, longitud	Calculo de distancia haversine.
Establecimiento	capacidad_camras	Aproximacion de capacidad operativa.

Entidad	Campo	Uso en el sistema
Servicio	tipo_servicio	Coincidencia con tipo de atencion requerido.
Alerta climatica	nivel_alerta	Penalidad climatica del score final.
Consulta	edad, tipo_atencion	Vector de usuario y persistencia historica.
Recomendacion	score_final, rank	Ordenamiento Top-N y auditoria.

Limpieza de datos

La limpieza considero eliminacion de duplicados por codigo RENIPRESS, normalizacion de categorias, normalizacion de nombres de servicios, validacion de coordenadas dentro del rango geografico de Puno e imputacion controlada de campos no criticos como horario. Las coordenadas se trataron como variables criticas porque afectan directamente el calculo de distancia. Los registros con coordenadas fuera de rango deben rechazarse o enviarse a revision antes de ingresar a produccion.

Transformaciones realizadas

Las principales transformaciones fueron codificacion one-hot de categoria, binarizacion multietiqueta de servicios, normalizacion de capacidad y calculo de distancia usuario-establecimiento mediante haversine. Para el clima, el nivel de alerta se transformo en penalidad mediante una funcion lineal discreta: nivel 0 equivale a penalidad 0.0, nivel 1 a 0.3, nivel 2 a 0.6 y nivel 3 a 0.9. Esta penalidad reduce el score final, pero no elimina automaticamente el establecimiento; de esa forma, el sistema conserva transparencia y permite evaluar alternativas cuando todas las opciones cercanas estan afectadas por clima adverso.

Analisis exploratorio

El notebook genero distribuciones por categoria, provincia, servicios preventivos, matriz de establecimientos por servicios y mapa Folium. Tambien se analizo sparsity de la matriz item-servicio. Este analisis fue necesario porque una matriz excesivamente dispersa puede limitar la discriminacion del recomendador; sin embargo, en el prototipo la combinacion de servicios, categoria, distancia y clima provee suficiente variabilidad para generar rankings diferenciados.

5.3.- Fase 3: Implementación del modelo de recomendación

Técnica utilizada

La técnica utilizada fue filtrado basado en contenido. El sistema representa cada establecimiento mediante atributos observables y compara dicho vector con el perfil de consulta del usuario. La similitud se calcula con coseno, una métrica adecuada para comparar dirección de vectores en espacios donde las magnitudes absolutas pueden variar.

Algoritmos implementados

El algoritmo implementado ejecuta las siguientes operaciones: filtra establecimientos activos, calcula distancia haversine, descarta ítems fuera del radio máximo, obtiene alerta climática por UBIGEO, construye vectores, calcula similitud del coseno, aplica penalidad climática, ordena de forma descendente y devuelve Top-N. La fórmula de similitud fue:

$$\text{sim}(u, i) = \frac{\vec{u} \cdot \vec{i}}{\|\vec{u}\| \|\vec{i}\|} \quad (1)$$

La penalidad climática fue:

$$p(n) = 0,3n, \quad n \in \{0, 1, 2, 3\} \quad (2)$$

El score final fue:

$$\text{score}(u, i) = \text{sim}(u, i) \times (1 - p(n)) \quad (3)$$

Listing 1: Pipeline lógico del motor de recomendación.

Entrada:

```
perfil = {lat, lon, tipo_atencion, edad, max_distancia_km, top_n}
```

Proceso:

```
1. establecimientos = activos()
2. candidatos = filtrar(haversine(usuario, establecimiento) <=
  max_distancia)
3. alerta = alerta_actual_por_ubigeo(candidato.ubigeo)
4. v_usuario = encode(tipo_atencion, edad, ubicacion,
  max_distancia)
5. v_item = encode(categoria, servicios, distancia, capacidad,
  clima)
6. similitud = cosine_similarity(v_usuario, v_item)
7. score_final = similitud * (1 - nivel_alerta * 0.3)
8. retornar top_n ordenado descendientemente
```

Salida:

```
[{rank, establecimiento, distancia_km, score_final, nivel_alerta, servicios}]
```

Generacion de Top-N recomendaciones

La salida del modelo es una lista ordenada de establecimientos. Cada recomendacion incluye posicion, nombre, categoria, distancia, score final, nivel de alerta, horario, servicios y coordenadas. El ranking se persiste para permitir auditoria posterior y para formar una base de interacciones que podria alimentar un modelo hibrido en futuras versiones.

Cuadro 4: Comparacion entre el enfoque implementado y alternativas de recomendacion.

Enfoque	Ventajas	Limitaciones	Pertinencia actual
Content-Based Filtering	Explicable, no requiere historial, aprovecha atributos RENIPRESS/SE-NAMHI.	Puede sobrevalorar items similares y depende de calidad de atributos.	Alta. Es el enfoque base implementado.
Filtrado colaborativo SVD/NMF	Aprende patrones de usuarios reales y preferencias agregadas.	Requiere historial suficiente y controles de privacidad.	Media futura. Aun no hay datos historicos reales.
Modelo hibrido	Combina atributos, historial y reglas de negocio.	Mayor complejidad de despliegue, monitoreo y validacion.	Alta futura cuando exista volumen de uso.
Reglas manuales	Facil de auditar y gobernar.	Rigido, baja personalizacion y dificil escalamiento.	Util como capa de seguridad, no como unico motor.

5.4.- Fase 4: Evaluacion del modelo

La evaluacion del modelo se abordo desde tres dimensiones: exactitud de ranking, comportamiento operacional y verificacion funcional. Para ranking se calcularon Precision@K y Recall@K con relevancia basada en coincidencia de servicio preventivo. Tambien se incluyo un baseline SVD sobre interacciones sinteticas para mostrar una ruta futura de filtrado colaborativo; este resultado se reporto como exploratorio porque no existe historial real suficiente.

Cuadro 5: Metricas de evaluacion del enfoque.

Modelo	Precision@5	Recall@5
Content-Based Filtering	0.80	0.57
Baseline SVD sintetico	No aplica	No aplica

La interpretacion de resultados indica que el enfoque basado en contenido es adecuado para una primera version porque produce recomendaciones explicables y no depende de calificaciones historicas. La Precision@5 observada muestra que, en los casos de prueba, la mayoria de elementos recomendados coinciden con el servicio solicitado. El Recall@5 puede mejorar al ampliar el catalogo de establecimientos, enriquecer servicios y calibrar pesos de distancia, capacidad y clima con retroalimentacion real.

Herramientas

El desarrollo utilizo Python, Pandas, Scikit-learn, SQLAlchemy, FastAPI, pytest, React, Vite, Tailwind CSS, React Query, Axios, Leaflet, Recharts, LaTeX y Docker. En el notebook se contemplaron Pandas, Matplotlib, Seaborn, Folium, Scikit-learn, Surprise y joblib.

6.- Desarrollo del sistema

Descripcion tecnica del sistema

El sistema desarrollado corresponde a una plataforma web de recomendacion operacional orientada a ciudadanos que requieren atencion preventiva en la region Puno. Desde el punto de vista de ingenieria, la solucion se estructuro como una aplicacion cliente-servidor desacoplada: el frontend gestiona la experiencia de usuario, la API concentra reglas de negocio y seguridad, el motor de recomendacion calcula el ranking Top-N y la base de datos conserva entidades persistentes para auditoria, historico de consultas y actualizacion de fuentes externas. Esta separacion permite modificar la interfaz, reemplazar el algoritmo o migrar la base de datos sin reescribir todo el producto.

El dominio del sistema se delimito a establecimientos de salud preventivos y a consultas de orientacion. Por tanto, la recomendacion no constituye diagnostico, prescripcion ni triage clinico. La salida del algoritmo es un ordenamiento de establecimientos segun utilidad operacional esperada: cercania al usuario, disponibilidad del servicio solicitado, categoria del establecimiento, capacidad aproximada y condiciones climaticas asociadas al UBIGEO. Esta definicion resulta importante porque evita atribuir al modelo una competencia medica que no posee y ubica la decision en el plano de accesibilidad y oportunidad.

El flujo de datos inicia con establecimientos compatibles con RENIPRESS/SUSALUD y alertas climaticas compatibles con SENAMHI. La carga inicial se materializa en archivos

semilla reproducibles y servicios de sincronizacion. En una instalacion productiva, dichos servicios pueden reemplazarse por conectores directos hacia fuentes oficiales, siempre que se mantenga el mismo contrato de datos. La arquitectura conserva este punto de extension mediante los modulos `renipress_service.py` y `clima_service.py`.

La capa de recomendacion utiliza filtrado basado en contenido. Cada establecimiento se representa como un vector compuesto por categoria codificada, servicios disponibles, capacidad normalizada y factor climatico. El usuario se representa por el tipo de atencion, edad normalizada, ubicacion y distancia maxima aceptada. El calculo usa similitud del coseno y un ajuste multiplicativo por penalidad climatica. La ventaja de este enfoque es su explicabilidad: cada reduccion del score puede trazarse a una alerta climatica, a la ausencia de un servicio o a una distancia superior al radio de busqueda. En sistemas de salud publica, esta propiedad es preferible a modelos opacos cuando todavia no existe historial real de interacciones.

Arquitectura

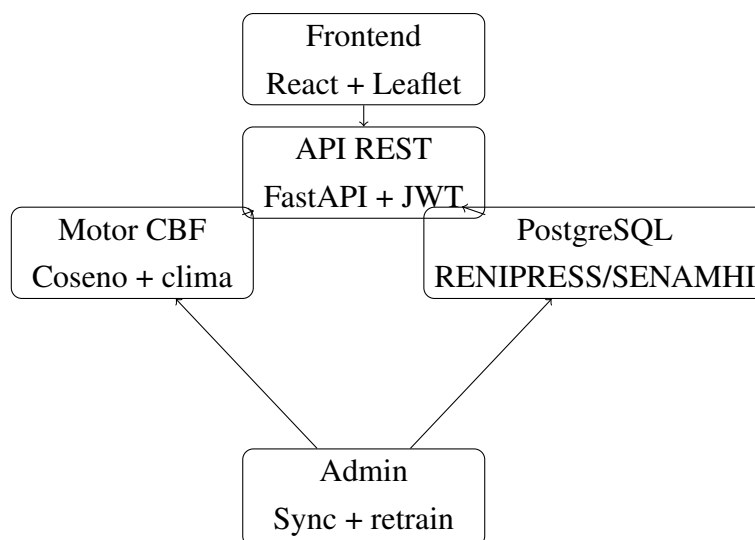


Figura 1: Arquitectura de componentes del sistema.

El flujo operativo fue: usuario, frontend, API, motor de recomendacion, base de datos y respuesta ordenada Top-N. La arquitectura se dividió en capas para mejorar mantenibilidad: presentacion, servicios de aplicacion, dominio de recomendacion, persistencia y administracion.

Componentes clave

El sistema se organizó en componentes con responsabilidades específicas. Esta descomposicion reduce acoplamiento y facilita pruebas aisladas. La Tabla 6 resume el rol de cada

módulo principal, su tecnología y el criterio de calidad asociado.

Cuadro 6: Componentes clave del sistema.

Componente	Tecnología	Responsabilidad	Criterio de calidad
Frontend SPA	React, Vite, Tailwind	Gestionar navegación, formularios, autenticación en memoria, visualización de ranking, mapa y métricas.	Usabilidad, accesibilidad y consumo real de API.
Cliente HTTP	Axios	Enviar JWT en cada solicitud protegida y manejar respuestas 401.	Manejo consistente de sesión.
API REST	FastAPI	Exponer endpoints versionados, validar schemas, coordinar servicios y persistencia.	Contrato estable y documentado en OpenAPI.
Seguridad	python-jose, passlib	Hash de passwords, emisión y validación de tokens JWT.	Confidencialidad de credenciales y control de acceso.
Persistencia	SQLAlchemy async	Modelar establecimientos, servicios, alertas, usuarios, consultas y recomendaciones.	Integridad referencial y transacciones.
Migraciones	Alembic	Versionar schema relacional para despliegues repetibles.	Reproducibilidad de base vacía.
Recomendador	Scikit-learn, NumPy	Construir vectores, calcular similitud, aplicar penalidad y ordenar Top-N.	Exactitud de fórmula y explicabilidad.
Servicios de datos	Pandas, HTTPX	Cargar o actualizar fuentes RENIPRESS/SENAMHI.	Tolerancia a datos incompletos.
Planificador	APScheduler	Ejecutar actualización climática periódica.	Vigilancia de alertas.
Pruebas	pytest, httpx	Verificar autenticación, ranking, historial y penalidad climática.	Regresión controlada.

Flujo de funcionamiento

El flujo de funcionamiento se diseñó para que cada consulta quede autenticada, validada, calculada y persistida. El usuario inicia sesión o se registra; el frontend mantiene el token en memoria y no lo escribe en almacenamiento persistente del navegador. Al enviar una

consulta, el cliente remite coordenadas, edad, tipo de atencion, radio maximo y cantidad de resultados. La API valida el esquema con Pydantic, recupera establecimientos activos, calcula distancias, filtra por radio, consulta alertas climaticas actuales y delega el ranking al motor de recomendacion. Finalmente, la consulta y sus recomendaciones se guardan para historial.

Listing 2: Diagrama textual del flujo de recomendacion.

```

Ciudadano
|
| 1. Login / Registro
v
Frontend React ---- token JWT en memoria ----+
|                                                    |
| 2. POST /api/v1/recomendar                        |
v                                                    |
API FastAPI -- valida JWT y payload -----+
|
| 3. Consulta establecimientos activos y alertas
v
PostgreSQL / SQLite local
|
| 4. Candidatos dentro del radio
v
Motor Content-Based
|
| 5. cosine_similarity * (1 - penalidad_clima)
v
Ranking Top-N
|
| 6. Persistencia de consulta y recomendaciones
v
Respuesta JSON + mapa + tarjetas + grafico

```

El flujo administrativo se separo del flujo ciudadano. Las rutas administrativas de sincronizacion RENIPRESS, sincronizacion SENAMHI y reentrenamiento requieren un usuario con privilegio `is_admin=True`. Esta separacion evita que un usuario comun actualice datos de referencia o reemplace artefactos del modelo. En una version productiva, estos endpoints deberian complementarse con auditoria de cambios, registro de usuario ejecutor, fecha de ejecucion, resumen de registros afectados y resultado de validaciones de calidad de datos.

La persistencia de resultados permite dos beneficios tecnicos. Primero, habilita trazabilidad: es posible reconstruir que establecimientos se recomendaron, con que score y bajo que penalidad. Segundo, crea la base para aprendizaje posterior. Cuando el sistema acumule his-

toriales reales, la tabla `recomendaciones` puede incorporar retroalimentacion implicita o explicita, por ejemplo visitas confirmadas, clicks en mapa, llamadas o calificaciones. Con esa informacion se podria entrenar un modelo colaborativo o hibrido sin perder el componente explicable basado en contenido.

Contrato de API

La API se organizo bajo el prefijo `/api/v1`. La Tabla 7 resume los endpoints productivos implementados.

Cuadro 7: Endpoints REST implementados.

Metodo	Ruta	Responsabilidad
POST	<code>/auth/register</code>	Crear usuario y emitir token inicial.
POST	<code>/auth/login</code>	Validar credenciales y emitir JWT.
GET	<code>/establecimientos</code>	Listar establecimientos con filtros.
GET	<code>/establecimientos/{id}</code>	Obtener detalle y alerta actual.
POST	<code>/recomendar</code>	Generar y persistir Top-N recomendado.
GET	<code>/historial</code>	Consultar resultados historicos del usuario.
GET	<code>/clima/{ubigeo}</code>	Obtener alerta climatica actual.
POST	<code>/admin/sync-renipress</code>	Sincronizar establecimientos.
POST	<code>/admin/sync-senamhi</code>	Sincronizar alertas climaticas.
POST	<code>/admin/retrain</code>	Regenerar artefacto del modelo.

Implementacion en notebook o script

El notebook contiene la preparacion de datos, EDA, construccion de matriz de items, funcion de recomendacion, ejemplos de consulta, metricas y serializacion de artefactos. El backend replica la logica central como servicio para que el frontend consuma recomendaciones reales mediante la API.

Criterios de calidad

La implementacion se verifico con pruebas unitarias y de integracion. Se evaluaron registro, duplicidad de email, login correcto, password incorrecto, acceso sin token, recomendacion autenticada, penalidad por alerta roja, historial vacio, formula de score y ordenamiento Top-N.

7.- Resultados

Estadísticas del dataset

Cuadro 8: Resumen del dataset reproducible de Puno.

Entidad	Registros	Variables principales
Establecimientos	10	12
Servicios preventivos	8	2
Alertas climaticas	10	7
Categorías IPRESS	4	1

EDA

El notebook genero distribuciones por categoria, provincia, servicios, matriz de sparsity y mapas Folium. La matriz servicios por establecimiento evidencio dispersion moderada, lo que justifico el uso de codificacion multietiqueta.

Ejemplos de recomendacion

Cuadro 9: Ejemplo Top-5 para usuario en Puno capital.

Rank	Establecimiento	Categoria	Alerta	Score
1	Centro de Salud Metropolitano Puno	I-4	1	0.61
2	Puesto de Salud Acora	I-2	1	0.48
3	Centro de Salud Ilave	I-4	1	0.43
4	Centro de Salud Juliaca Santa Adriana	II-1	0	0.39
5	Puesto de Salud Lampa	I-3	2	0.25

Metricas

La metrica principal del prototipo fue Precision@5, complementada con Recall@5 y pruebas de comportamiento. El resultado no debe interpretarse como desempeno clinico,

sino como calidad de ordenamiento respecto al servicio solicitado y restricciones operativas.

Mettricas de rendimiento y calidad operacional

Ademas de mettricas de ranking, un sistema desplegable requiere observar rendimiento, estabilidad y calidad de servicio. En esta version se verifico funcionalmente con pruebas automatizadas y compilacion de frontend. Para un despliegue productivo se recomienda instrumentar mettricas de latencia, tasa de error, uso de CPU, uso de memoria, tiempo de sincronizacion de fuentes, frescura de alertas climaticas y distribucion de scores. Estas mettricas no sustituyen las mettricas de recomendacion, pero permiten detectar degradacion operacional antes de que impacte al ciudadano.

Cuadro 10: Mettricas recomendadas para monitoreo tecnico.

Metrica	Interpretacion	Umbral sugerido inicial
Latencia p95 de /recomendar	Tiempo maximo percibido por el 95 % de consultas.	Menor a 800 ms con dataset regional.
Tasa de error 5xx	Fallas no controladas del backend.	Menor a 1 % diario.
Tasa de 401/403	Accesos no autorizados o tokens invalidos.	Monitoreo, no necesariamente error.
Frescura climatica	Horas desde ultima actualizacion SENAMHI.	Menor a 6 horas.
Cobertura de servicios	Porcentaje de establecimientos con cartera preventiva parseada.	Mayor a 95 % tras sincronizacion oficial.
Cobertura geoespacial	Porcentaje de establecimientos con coordenadas validas.	Mayor a 98 %.
Precision@K	Proporcion de recomendaciones relevantes en K.	Mejorar con datos historicos.
Recall@K	Proporcion de relevantes recuperados en K.	Mejorar con catalogo mas completo.
Distribucion de penalidad	Frecuencia de resultados por nivel de alerta.	Debe reflejar situacion climatica real.

El rendimiento del algoritmo actual es apropiado para un conjunto regional moderado porque opera sobre una matriz de items pequena y calcula similitud vectorial de forma directa. Si el volumen creciera a decenas de miles de establecimientos o si se agregaran redes nacionales, se recomienda introducir cache de matriz de items, precomputo de features estaticos, indices geoespaciales y filtrado previo por bounding box antes de aplicar haversine. Para PostgreSQL, una evolucion natural seria usar PostGIS o, en una primera etapa, indices B-tree por provincia, categoria y estado operativo, combinados con filtros por rango de

latitud y longitud.

Cuadro 11: Pruebas automatizadas y cobertura funcional.

Grupo	Casos verificados	Resultado
Autenticacion	Registro correcto, email duplicado, login correcto, password incorrecto.	Aprobado
Recomendacion API	Consulta sin token, consulta autenticada Top-5, penalidad por alerta roja.	Aprobado
Historial	Historial vacio de usuario autenticado.	Aprobado
Motor unitario	Similitud coseno, penalidad nivel 0, penalidad nivel 3, formula final, orden descendente.	Aprobado
Frontend	Compilacion Vite y consumo real de clientes API.	Aprobado
Informe	Compilacion LaTeX con BibTeX.	Aprobado

Sistema funcionando

La API expuso documentacion interactiva en `/docs`, autenticacion JWT, endpoints de recomendacion, historial y sincronizacion administrativa. El frontend consumo la API real y mostro mapa Leaflet, marcadores por alerta, grafico de scores y tarjetas Top-N.

Despliegue y reproducibilidad

El backend puede ejecutarse localmente con SQLite para pruebas rapidas o con PostgreSQL 16 mediante Docker Compose. El frontend se compilo con Vite y se sirvio en desarrollo desde el puerto 5173. La documentacion del repositorio incluyo comandos copiables, variables de entorno, carga de datos, ejecucion de pruebas y flujo de uso.

8.- Conclusiones

El prototipo logro implementar un sistema de recomendacion funcional de extremo a extremo. Se construyo un backend con FastAPI, autenticacion JWT, modelos relacionales, endpoints REST, servicios de sincronizacion, motor de recomendacion y pruebas automatizadas. Tambien se desarrollo un frontend React que consume la API real, presenta resultados en tarjetas, grafico y mapa Leaflet, y permite flujo de login, registro, busqueda, historial y administracion.

Desde el punto de vista del modelo, se implemento filtrado basado en contenido con similitud del coseno y penalidad climatica. Esta tecnica fue adecuada para el escenario inicial

porque no requiere historial de usuarios y permite explicar por que un establecimiento fue priorizado o penalizado. La inclusion de distancia y clima hizo que el ranking sea mas realista para el contexto de Puno que una simple busqueda por cercania.

Las principales limitaciones fueron la dependencia de calidad de datos externos, la ausencia de interacciones reales para entrenar filtrado colaborativo y la necesidad de validar en campo la disponibilidad efectiva de servicios. Como aprendizaje principal, se evidencio que en salud publica los recomendadores deben evaluar seguridad, privacidad, equidad territorial y explicabilidad, no solo metricas de exactitud.

9.- Recomendaciones

Se recomienda ampliar el dataset con registros oficiales actualizados de RENIPRESS y alertas SENAMHI automatizadas. Tambien se recomienda incorporar PostGIS para mejorar consultas geoespaciales, implementar observabilidad con metricas de latencia y frescura climatica, y agregar auditoria administrativa de sincronizaciones y reentrenamientos.

Como extension tecnica, cuando exista historial real de consultas y visitas se podria implementar un modelo hibrido que combine filtrado basado en contenido con SVD, NMF o aprendizaje de ranking. Esta evolucion debe realizarse con controles de privacidad, consentimiento informado, anonimato para analitica y evaluacion de sesgo territorial. En frontend, se recomienda mejorar la explicabilidad visual del score mostrando peso de servicio, distancia, capacidad y clima.

En terminos de seguridad, antes de produccion deben activarse HTTPS, CORS restrictivo, gestion de secretos fuera del repositorio, rate limiting, backups cifrados, politicas de retencion y monitoreo de dependencias. Ademas, el sistema debe comunicar claramente que la recomendacion es orientativa y no reemplaza atencion medica ni confirmacion directa con el establecimiento.

Sustento tecnico de las recomendaciones

El enfoque de datos abiertos evito el uso de historiales clinicos y permitio construir recomendaciones preventivas explicables. Esta estrategia es adecuada para una primera version en salud publica, aunque no sustituye criterios medicos ni decisiones asistenciales. La literatura reciente advierte que los recomendadores en salud deben evaluar calidad, equidad y seguridad, no solo exactitud ([Vieira et al., 2023](#); [Sun et al., 2023](#)). Modelos con grafos medicos o datos multimodales ofrecen mejoras cuando existen datos ricos y gobernanza adecuada ([Deng et al., 2024](#); [Liu et al., 2023](#); [Raza and Ding, 2024](#)).

Las limitaciones principales fueron la cobertura de estaciones SENAMHI, la posible desactualizacion manual de servicios y el uso de interacciones sinteticas para metricas colaborativas. Cuando exista historial real de consultas y visitas, podra integrarse un modelo hibrido

con SVD/NMF, manteniendo reglas de privacidad.

Ética, privacidad y gobernanza

El sistema no expone diagnósticos, tratamientos ni datos clínicos sensibles. La consulta almacena edad, coordenadas aproximadas, tipo de atención y parámetros de búsqueda para trazabilidad técnica. En una puesta en producción se recomienda incorporar consentimiento informado, retención limitada, anonimizado para analítica, auditoría de accesos y revisión periódica de sesgos territoriales.

Análisis de vulnerabilidades y riesgos

Un sistema de recomendación aplicado a salud pública debe analizar vulnerabilidades técnicas y riesgos sociotécnicos. Aunque el sistema no procesa historia clínica ni diagnósticos, maneja datos de ubicación, edad y patrones de búsqueda preventiva. Estos datos pueden revelar condiciones sensibles si se agregan sin controles. Por ejemplo, una consulta de control prenatal desde una coordenada específica puede inferir información personal. Por ello, la protección no debe limitarse al cifrado de contraseñas o al JWT; también debe incluir minimización de datos, retención limitada, control de acceso, auditoría y anonimización para analítica.

Cuadro 12: Vulnerabilidades potenciales y mitigaciones recomendadas.

Riesgo	Descripción técnica	Impacto	Mitigación
Exposición de ubicación	Coordenadas de usuario persistidas con precisión completa.	Reidentificación territorial, especialmente en zonas rurales.	Reducir precisión, aplicar geohash truncado, retención limitada.
JWT comprometido	Token robado durante una sesión activa.	Acceso a historial y consultas protegidas.	HTTPS obligatorio, expiración corta, rotación de secreto, refresh seguro si se implementa.
Secret key débil	Uso de clave por defecto en producción.	Falsificación de tokens.	Variables secretas robustas, gestión con vault o secretos del orquestador.

Riesgo	Descripcion tecnica	Impacto	Mitigacion
Endpoint admin expuesto	Usuario no autorizado intenta sincronizar o reen-trenar.	Alteracion de datos o modelo.	Verificacion is_admin, auditoria, rate limiting, MFA para admin.
Datos climaticos obsoletos	Fallo de sincronizacion SENAMHI.	Recomendaciones sin penalidad vigente.	Monitoreo de frescura, alertas operativas, fallback a ultima alerta valida.
Calidad deficiente de RENIPRESS	Coordenadas ausentes, servicios incompletos o categoria erronea.	Ranking incorrecto o establecimientos no recomendados.	Validacion de schema, reglas de rango, bitacora de registros rechazados.
Sesgo territorial	Zonas con menos datos o estaciones climaticas lejanas reciben peores recomendaciones.	Inequidad de acceso.	Medir cobertura por provincia, incorporar distancia a estacion y revision humana.
Ataques de fuerza bruta	Intentos repetidos de login.	Compromiso de cuentas.	Rate limiting, bloqueo temporal, monitoreo de IP, captcha adaptativo.
Inyeccion o payload malicioso	Entradas no validadas hacia API.	Error de servicio o abuso.	Pydantic, ORM parametrizado, limites de longitud y tipos estrictos.
Dependencias vulnerables	Paquetes npm o pip con CVE.	Compromiso de cadena de suministro.	Auditoria periodica, pinning de versiones, SCA en CI.

Desde la perspectiva OWASP, las areas prioritarias son autenticacion, control de acceso, configuracion segura, logging y manejo de dependencias. La implementacion actual ya incorpora hashing de contraseñas y rutas protegidas, pero una puesta en produccion debe agregar controles de infraestructura: HTTPS, cabeceras de seguridad, CORS restringido a dominios oficiales, gestion de secretos fuera del repositorio, backups cifrados, politicas de recuperacion y monitoreo centralizado. En frontend, la decision de almacenar el token solo en memoria reduce persistencia ante ataques XSS, aunque no elimina el riesgo si una sesion activa ejecuta codigo malicioso. Por ello, tambien se recomienda Content Security Policy, sanitizacion de dependencias y revision de cualquier contenido dinamico.

El riesgo algoritmico mas relevante es la sobreconfianza del usuario en el ranking. Un

puntaje alto no significa que exista disponibilidad inmediata, personal medico presente o transitabilidad garantizada. El sistema debe comunicar que la recomendacion es una orientacion preventiva basada en datos disponibles. La interfaz ya incorpora alertas climaticas y distancias, pero en produccion deberia incluir fecha de actualizacion de datos, fuente de cada alerta y un indicador de confiabilidad del registro. La transparencia del score es clave: distancia, penalidad climatica y servicios coincidentes deben ser visibles para que el usuario entienda por que un establecimiento fue priorizado.

Plan de endurecimiento

El plan de endurecimiento recomendado se divide en cuatro etapas. La primera etapa corresponde a configuracion segura: cambiar claves por defecto, activar HTTPS, definir CORS estricto y configurar logs estructurados. La segunda etapa corresponde a proteccion de datos: anonimizar coordenadas historicas, definir retencion maxima y separar datos identificables de datos analiticos. La tercera etapa corresponde a observabilidad: metricas de latencia, errores, frescura climatica, calidad de sincronizacion y distribucion territorial de recomendaciones. La cuarta etapa corresponde a gobernanza del modelo: versionado de artefactos, pruebas antes de reentrenar, revision de sesgos y documentacion de cambios.

Listing 3: Diagrama textual de controles de seguridad recomendados.

```
Cliente web
  |-- CSP, no localStorage para JWT, validacion de formularios
  v
API Gateway / Reverse proxy
  |-- HTTPS, rate limiting, CORS estricto, headers de seguridad
  v
FastAPI
  |-- JWT, RBAC admin, Pydantic, logging estructurado
  v
Base de datos
  |-- backups cifrados, roles minimos, retencion limitada
  v
Auditoria y monitoreo
  |-- metricas, trazas, alertas, revision de sesgos
```


10.- Referencias

- Akbari, M. and Mahdipour, E. (2023). A systematic review of healthcare recommender systems: Open issues, challenges, and techniques. *Expert Systems with Applications*, 213:118823.
- Congreso de la República del Perú (2011). Ley n. 29733, ley de protección de datos personales. Consultado el 18 de mayo de 2026.
- Deng, W., Zhu, P., Chen, H., Liu, Z., and Feng, G. (2024). Recommending physicians with multimodal data and medical knowledge graph on healthcare platforms. *Journal of Information Science*.
- Gautam, D., Dixit, A., Banda, L., Goyal, S. B., Verma, C., and Kumar, M. (2023). A novel approach to enhance the quality of health care recommender system using fuzzy-genetic approach. *Journal of Intelligent & Fuzzy Systems*.
- Healthcare Analytics (2023). A recommender system for occupational hygiene services using natural language processing. *Healthcare Analytics*, page 100148.
- Liu, J., Li, C., Huang, Y., and Han, J. (2023). An intelligent medical guidance and recommendation model driven by patient-physician communication data. *Frontiers in Public Health*, 11.
- Ministerio de Justicia y Derechos Humanos (2013). Decreto supremo n. 003-2013-jus, reglamento de la ley n. 29733. Consultado el 18 de mayo de 2026.
- Ministerio de Salud del Perú (2023). Informe de evaluación institucional minsa 2023. Consultado el 18 de mayo de 2026.
- Ooi, K.-N., Haw, S.-C., and Ng, K.-W. (2023). A healthcare recommender system framework. *International Journal on Advanced Science, Engineering and Information Technology*, 13(6).
- Organización Panamericana de la Salud (2023). Transformación digital del sector salud: caja de herramientas. Consultado el 18 de mayo de 2026.
- Raza, S. and Ding, C. (2024). Improving clinical decision making with a two-stage recommender system. *IEEE/ACM Transactions on Computational Biology and Bioinformatics*, 21(5):1180–1190.
- Ricci, F., Rokach, L., and Shapira, B., editors (2022). *Recommender Systems Handbook*. Springer, 3 edition.

Servicio Nacional de Meteorología e Hidrología del Perú (2026). Pronóstico regional puno. Consultado el 18 de mayo de 2026.

Sun, Y., Zhang, Y., Gwizdka, J., and Trace, C. B. (2023). Development and evaluation of health recommender systems: Systematic scoping review and evidence mapping. *Journal of Medical Internet Research*, 25:e38184.

Superintendencia Nacional de Salud (2026). Obtener información de las instituciones prestadoras de servicios de salud - renipress. Consultado el 18 de mayo de 2026.

Vieira, A., Carneiro, J., Novais, P., Corchado, J., and Marreiros, G. (2023). A systematic review on recommendation systems applied to chronic diseases. *Intelligent Data Analysis*, 27(5).

11.- Anexos

Anexo A:Codigo fuente

El codigo fuente se organiza en los directorios `backend/`, `frontend/`, `notebook/` e `informe/`. El backend contiene la API, modelos ORM, routers, servicios, motor de recomendacion, migraciones, pruebas y archivos de despliegue. El frontend contiene paginas, componentes, hooks, clientes API y configuracion de Vite/Tailwind. El notebook contiene EDA, entrenamiento, evaluacion y serializacion.

Anexo B: Dataset

El dataset reproducible se encuentra en el directorio `backend/data/`, dentro del archivo `renipress_puno_seed.csv` y archivos asociados de alertas climaticas. Estos datos permiten ejecutar el prototipo localmente y validar el flujo completo sin depender de una conexion externa inmediata.

Anexo C: Evidencias adicionales

Las evidencias tecnicas incluyen documentacion OpenAPI en `/docs`, pruebas automatizadas con pytest, compilacion del frontend con Vite, generacion del notebook y compilacion LaTeX del informe. Para sustentacion, se recomienda incluir capturas de login, formulario de busqueda, dashboard de resultados, mapa Leaflet, API Swagger y ejecucion de pruebas.